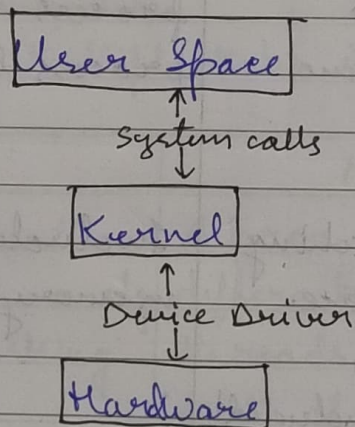


Kernel: A Kernel is that part of the operating system which interacts directly with the hardware and performs the most crucial tasks.

- a) Heart of OS/ Core component
- b) Very first part of OS to load on start-up.

User space: Where application software runs, apps don't have privileged access to the underlying hardware. It interacts with kernel.

- a) GUI → Graphical User Interface
- b) CLI → Command Line Interface



Functions of Kernel:

1) Process Management:

- a) Scheduling processes & threads on the CPUs.
- b) Creating & deleting both user and system process.
- c) Suspending & resuming process.
- d) Providing mechanism for process synchronization or process communication.

27 File management:

- Creating & deleting files
- Creating & deleting directories to organize file.
- Mapping files into secondary storage.
- Backup support onto a stable storage media.

37 Memory management:

- Allocation & deallocation of memory space as per need.
- Keeping track of which part of memory are currently being used and by which process.

47 I/O management: To manage & control I/O operations and I/O devices.

a7 Buffering (copying of data b/w two devices), caching and spooling.

17 Spooling (Simultaneous Peripheral Operations Online)

It is a process in which data is temporarily held to be used and executed by a device, program or system at a later time.

- Within differing speed two jobs
- E.g.: Print spooling and mail spooling.

117 Buffering

- Within one job.

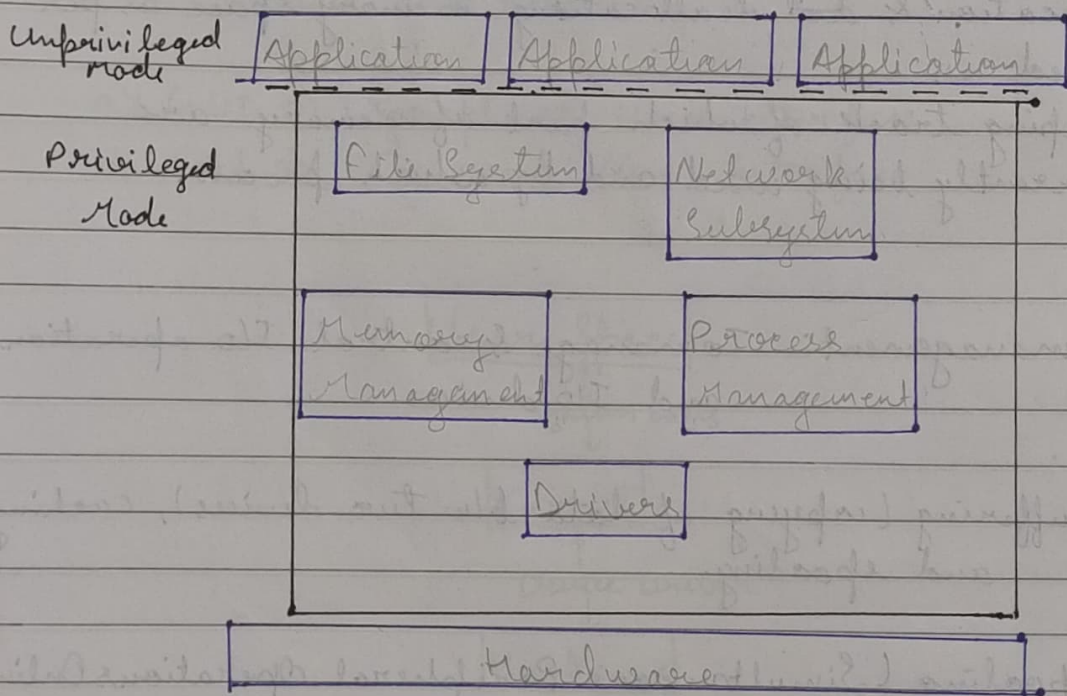
- E.g:- Youtube video buffering

iii) Caching

- Memory caching, web caching etc.

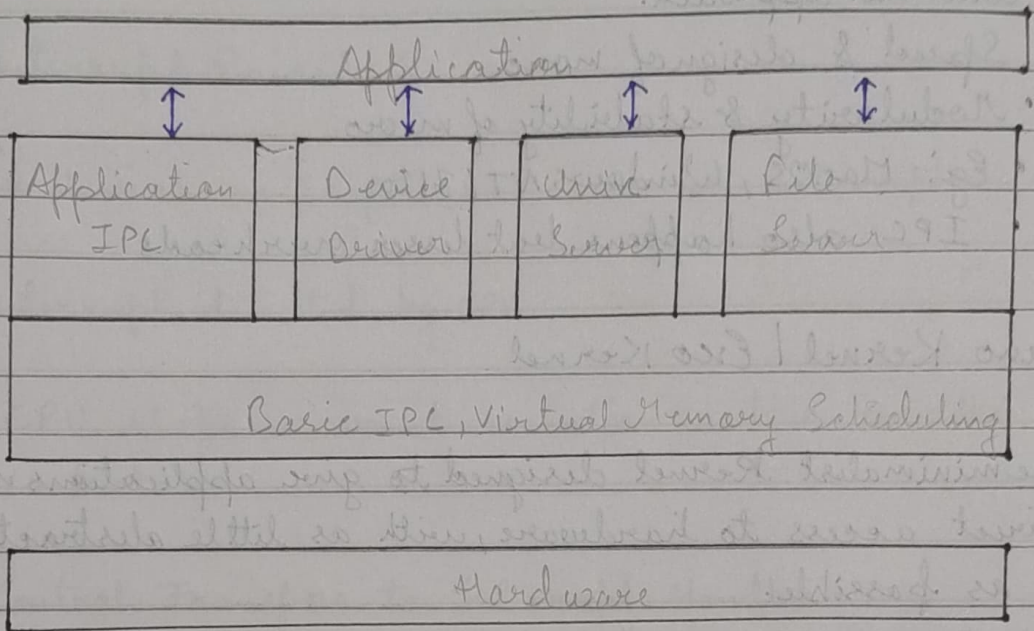
Types of Kernels:

1. Monolithic Kernel



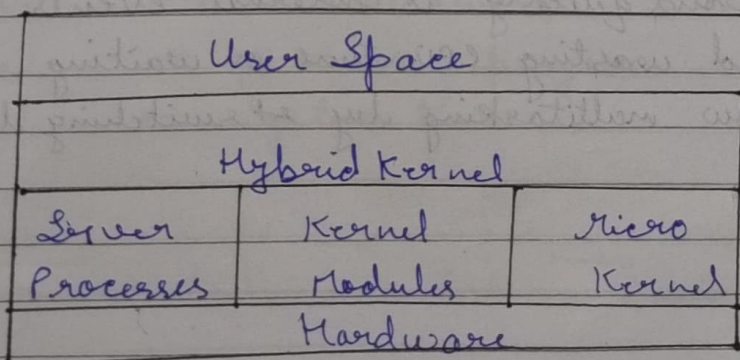
- All functions are in Kernel itself.
- Bulky in size.
- Memory required to run is high.
- Less reliable, one module crashes $\rightarrow$ whole Kernel down.
- High performance as communication is fast. (Less user mode, Kernel mode overheads)
- Eg:- Linux, Unix, MS-DOS.

27 Micro Kernel



- Only major functions are in Kernel.
 - memory management
 - Process management
- File management & I/O are in User Space
- Smaller in size
- More Reliable
- More Stable
- Performance is slow
- Overhead switching b/w user mode & Kernel mode.
- E.g:- 4.4 linux, Symbian OS, MINIX etc.

37 Hybrid Kernel:



- Advantages of both worlds.
- Combined approach.
- Speed & design of mono.
- Modularity & stability of micro
- Eg:- MacOS, Windows NT/7/10
- IPC also happens but lesser overheads

4) Nano Kernel / Exo Kernel

The minimalist Kernel designed to give applications more direct access to hardware, with as little abstraction as possible.

Goal: Maximum performance & flexibility by avoiding unnecessary OS-level decision.

Instead of providing high-level abstractions, the exo Kernel securely exposes hardware resources.

Interrupt Handling:-

An interrupt is a signal sent to the CPU from hardware or software that temporarily pauses the current process so the CPU can handle something more urgent.

Why are interrupts needed?

To respond quickly to external events

To avoid wasting CPU time on waiting

To allow multitasking by switching b/w process

Steps in Interrupt Handling:

1) Interrupt Raised

- A device or software raises an interrupt.

2) Interrupt detected by CPU

- CPU stops current execution and saves the context.

3) Control Transfers to Interrupt Handler

- CPU jumps to a specific function called the interrupt service routine (ISR) or handler.

4) Interrupt is Handled

- ISR performs the necessary task.

5) Resume Original Execution

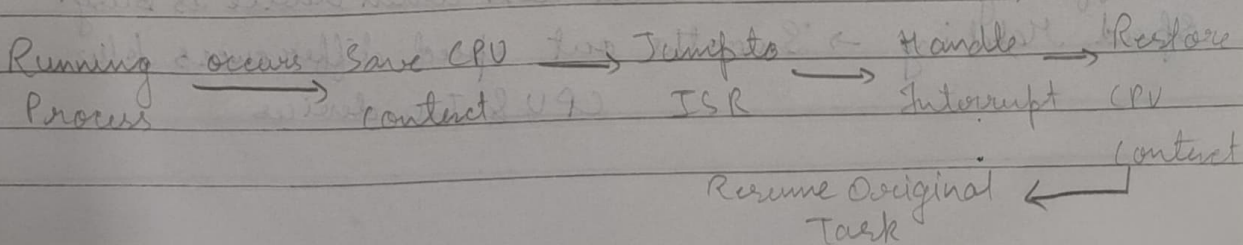
- After handling, CPU restores the saved context and resumes what was doing before the interrupt.

Types of Interrupts:-

Hardware → External devices

Software → OS or user program

Timer → System timer



Q7) How will communication happen b/w user mode and Kernel mode?

→ Inter process communication

- Two processes executing independently, having independently memory space, But some may need to communicate to work.
- Done by shared memory and message passing

IPC refers to mechanisms an operating system provides to allow processes to communicate with each other and synchronize their actions. Since processes in modern operating systems run in separate memory spaces, IPC is essential for data exchange and coordination b/w them

Why IPC is Needed:

- Data sharing b/w processes
- Synchronization
- Resource sharing
- Event notification
- Modularity in system design

Spooling:-

Concurrency → Multiple users/processes can queue I/O tasks
Efficiency → Devices aren't kept waiting for input
Buffering → Avoids data loss when device is busy.
Speed Matching → Smooths out speed differences b/w CPU & I/O devices

PCB

Stores information about a process. It helps the OS manage processes.

PCB is like your passport. It holds your identity, travel details, and status.

Spooling

Handles I/O operations, especially with slow devices, by queuing tasks in buffers for later execution.

Spooling is like standing in queue at the airport check-in.